Etapa 9: ¿agrupar columnas ayuda?

La etapa anterior mostró que la factorización concentra gran parte del tiempo. Estudiamos ahora si cambiar la propiedad de las columnas mejora el flujo completo. Se mantiene right-looking escalar: cada columna se termina y difunde antes de continuar.

Hipótesis

Un bloque de b columnas consecutivas se asigna por turnos a cada proceso. Agrupar columnas puede modificar la localidad y las esperas, pero también el reparto del trabajo. Como las columnas locales ya estaban empaquetadas por filas, b mayor no garantiza mejor acceso a memoria.

$$\text{propietario}(j) = \lfloor j/b \rfloor \bmod P$$

Con P=3 y b=2: proceso 0 recibe 0,1,6,7; proceso 1 recibe 2,3,8,9; proceso 2 recibe 4,5,10,11. b=1 reproduce los propietarios de la distribución cíclica previa.

Qué cambia y qué se conserva

Cambia el mapa global/local y la cantidad de columnas de cada proceso. Se conserva la matriz generada, el kernel de actualización con vector local, n difusiones por factorización y L reunida en rango 0. Todas las variantes usan el mismo código general de índices, incluido b=1. No comparamos contra cifras de campañas anteriores.

Esto es distribución por bloques de columnas. No es Cholesky por bloques con operaciones matriz-matriz ni agrupa las comunicaciones: sigue habiendo una difusión por columna.

Protocolo y reloj

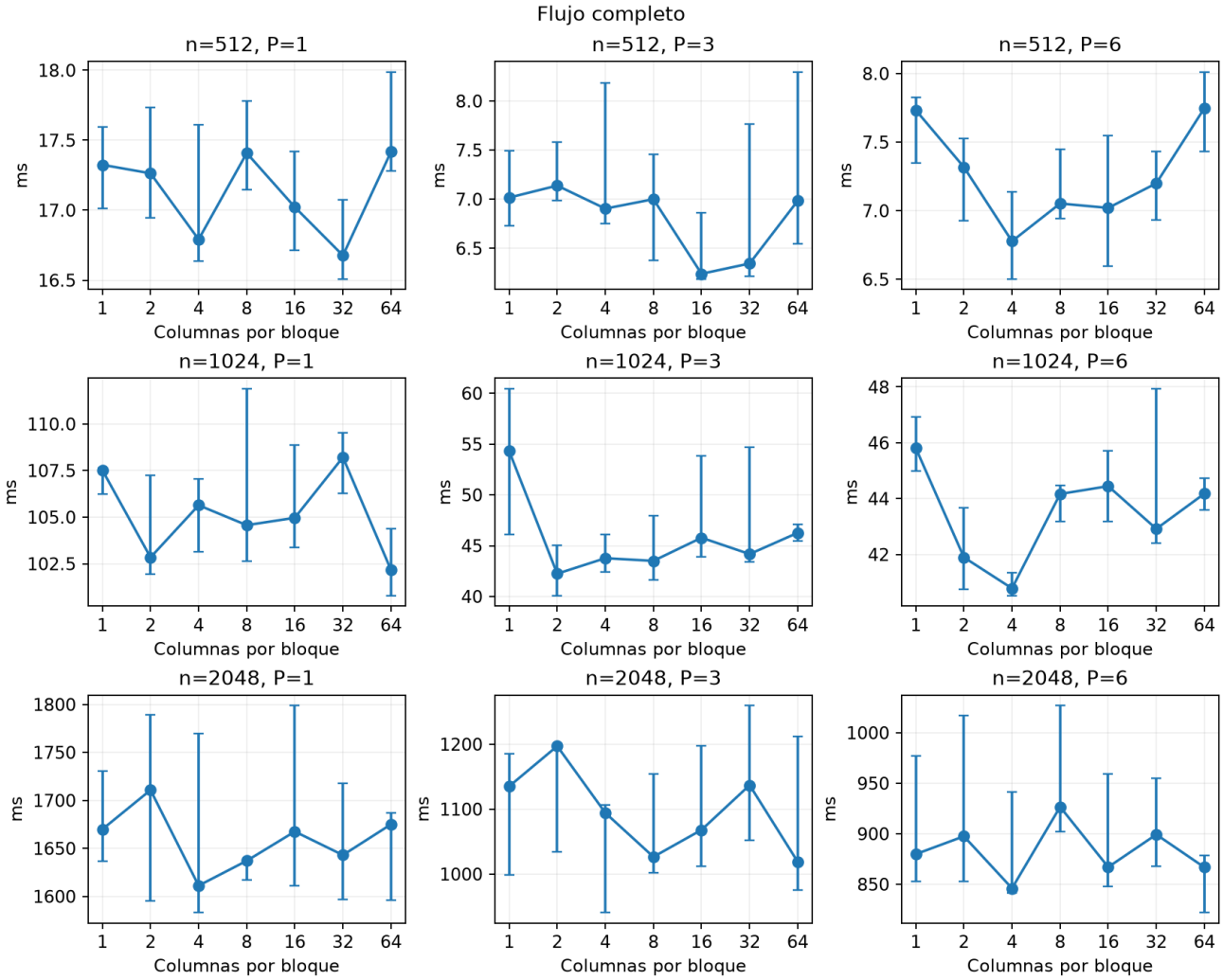
n=512,1024,2048; P=1,3,6; b=1,2,4,8,16,32,64. Un calentamiento por variante y siete repeticiones. El orden rota para que cada bloque ocupe cada posición una vez; no balancea todas las parejas de vecinos. Se rota P entre tamaños. Total: 441 observaciones en una sesión local WSL, afinidad por núcleo, GCC -O3, semilla 2026.

Total principal: reloj de rango 0 desde reserva y preparación del reparto hasta generación distribuida, factorización y recolección completa. Excluye lanzamiento/cierre MPI, referencias, validación y liberación. Las fases incluyen esperas. Se compara la mediana de totales medidos directamente.

Control con un proceso

Con P=1, las columnas y operaciones son iguales para todos los tamaños de bloque. Diferencias grandes en ese control requieren cautela: no pueden atribuirse a balance entre procesos. El mapa general introduce aritmética distinta de las versiones históricas.

Tiempo hasta obtener el resultado



Puntos: medianas; barras: Q1-Q3, no intervalos de confianza. Cada panel tiene su propia escala vertical. Comparamos bloques dentro de cada configuración.

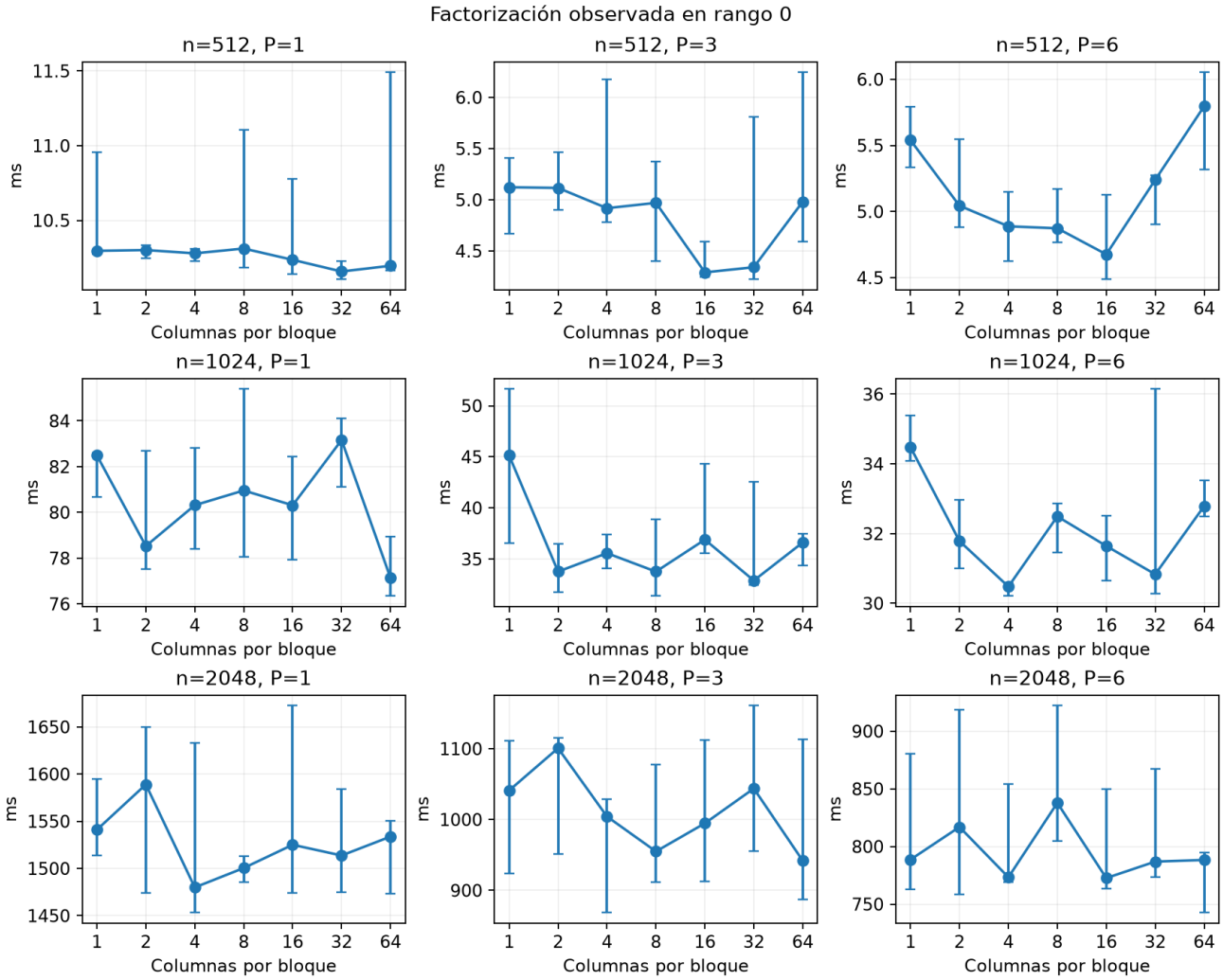
Interpretación

El total contempla los cambios de generación y desempaquetado provocados por el reparto, además del kernel. Una menor mediana aislada es un candidato para repetir, no una demostración de superioridad universal. Elegir el mínimo entre siete alternativas puede favorecer fluctuaciones de esta sesión.

Reproducibilidad

Campaña: resultados/bloques_local_20260922. Implementación: src/compare_blocks.c y mapas de src/cholesky_mpi.c. Ejecución: scripts/comparar_bloques.sh; análisis: scripts/analizar_bloques.py. Código, entorno y fuentes congeladas acompañan los resultados. Desarrollo y texto con asistencia de IA; datos obtenidos ejecutando el código localmente.

Factorización y reparto del trabajo



Factorización observada por rango 0: incluye sus esperas de comunicación. No es una medición separada de cómputo puro ni del máximo de cada proceso.

$$W_r = \sum_{k \in C_r} k(n-k), \qquad I = \frac{\max_r W_r}{\sum_r W_r / P}$$

Cada columna k recibe k actualizaciones sobre n-k elementos. I=1 indica igualdad en este conteo acumulado; no garantiza balance en cada etapa, ni incluye pivotes, indexación o comunicaciones.

n=2048, P=6: b=1: 1.0000, b=2: 1.0000, b=4: 1.0000, b=8: 1.0002, b=16: 1.0006, b=32: 1.0026, b=64: 1.0090.

Decisión provisional y siguiente pregunta

n	P	Mejor b observado	b=1 (ms)	Mejor (ms)	Cociente
512	3	16	7.02	6.24	1.124x
512	6	4	7.73	6.78	1.141x
1024	3	2	54.32	42.24	1.286x
1024	6	4	45.81	40.78	1.123x
2048	3	64	1135.03	1018.87	1.114x
2048	6	4	879.80	845.97	1.040x

Estos mínimos son descriptivos y fueron elegidos después de medir. Seleccionamos b=4 para una repetición con P=6, manteniendo b=1 como referencia. No se selecciona un tamaño universal.

Repetición del candidato

n=1024, P=6: b=1 tarda 48.93 ms y b=4 tarda 44.02 ms; cociente 1.112x. Son nuevas ejecuciones, con siete repeticiones por bloque, en la misma sesión local.

n=2048, P=6: b=1 tarda 931.67 ms y b=4 tarda 929.70 ms; cociente 1.002x. Son nuevas ejecuciones, con siete repeticiones por bloque, en la misma sesión local.

Esta repetición añade 98 observaciones. Para n=1024 se mantiene una reducción cercana al 10 % con b=4; para n=2048 la diferencia cae a aproximadamente 0,2 %, insuficiente para sostener una ventaja práctica estable. Conservamos b=1 como referencia y b=4 como candidato para n=1024, P=6. No adoptamos b=4 universalmente.

Validación

Se aceptaron 441 observaciones. Máximo residuo proyectado: 1.611e-15; máxima diferencia respecto del factor secuencial: 0.000e+00. Se comprueban propietarios, cantidad de columnas, prefijos e índices; b=1 se compara con el mapa cíclico original. Casos pequeños: n=1,2,3,7,31,65 con P=1,3,6, incluyendo bloques incompletos y procesos vacíos.

Las matrices grandes se contrastan elemento a elemento con el factor secuencial y con ocho proyecciones. Los casos pequeños también usan residuo Frobenius completo. El análisis Python verifica el conteo de trabajo mediante una enumeración independiente. El residuo proyectado por sí solo no es un certificado.

Próxima pregunta

¿La ventaja se mantiene en otras sesiones y matrices? Si la distribución aporta poco, corresponde investigar reutilización de datos dentro del cálculo, mediante una referencia LAPACK y, según el alcance, Cholesky por bloques. El reparto por bloques no sustituye esa optimización.